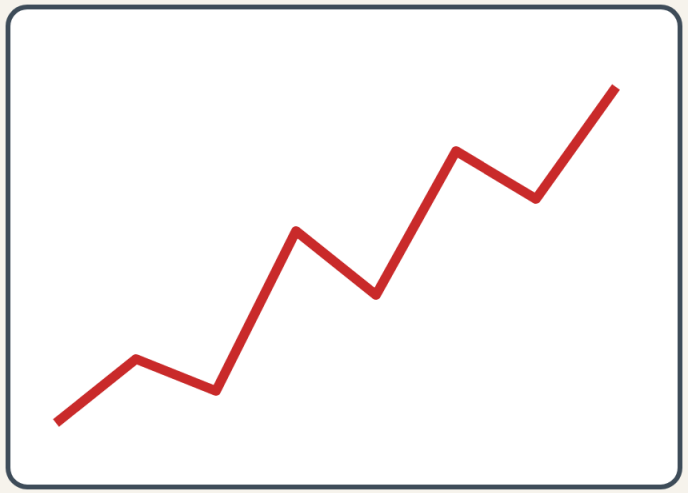
LEZIONE 5

Monitoraggio e Prometheus

ITS CyberSecurity - Jesi (AN) 2026/2027

Corso Logging, Monitoring, Troubleshooting



un numero nel tempo

prof. Luca Ferroni

Log o metrica?

Log

```
sshd[812]: Failed password
for root from 45.9.14.207
```

racconta un evento

parole, una riga per fatto

Metrica



misura un numero

CPU al 37%, ogni 15 secondi

Classificazione degli indicatori chiave

Cosa misurare, a seconda di cosa si osserva

Apparati di rete

router, switch, firewall

- throughput
- errori sulle interfacce
- latenza
- CPU e RAM

di solito via SNMP, senza agent

Server

GNU/Linux, Windows

- utilizzo CPU
- RAM
- I/O disco
- spazio disco
- processi attivi

con un agent (Alloy, node exporter)

Servizi e applicazioni

web, database, API

- uptime
- status code HTTP
- tempi di risposta
- porte aperte

controllati anche da fuori (curl, nc)

Una serie temporale

Nome + etichette + tante coppie (istante, valore)

```
node_load1 {host="mario-rossi"}
```

il nome: il carico del PC

le etichette: di chi è

istante	valore
10:00:00	0.42
10:00:15	0.51
10:00:30	0.47
10:00:45	0.63

Ogni 15 secondi una riga in più.

Database a serie temporali

TSDB, in inglese: time series database

- Aggiunge numeri in fondo, in ordine di tempo.
- Si legge per intervalli: ultimi 5 minuti, ultima ora.
- Prometheus ne ha uno dentro: la cartella /prometheus.

Non si cancella riga per riga: si buttano via i giorni vecchi.

Retention

Per quanto tempo tengo i dati?

```
--storage.tsdb.retention.time=15d
```

- Più giorni = più disco.
- Dopo 15 giorni i dati più vecchi spariscono.
- Vale anche per i log, in Loki.

Tenere d'occhio il disco

Log e metriche crescono ogni giorno

Misurare

```
$ df -h /
$ docker system df
$ sudo du -sh /var/lib/docker/volumes/*
$ journalctl --disk-usage
```

Limitare (retention)

```
# Prometheus (compose.yml)
--storage.tsdb.retention.time=15d

# Loki (config.yml)
limits_config:
  retention_period: 744h
compactor:
  retention_enabled: true
  delete_request_store: filesystem

# journald (/etc/systemd/journald.conf)
SystemMaxUse=500M
```

Retention giusta =

quanto indietro devo poter guardare

e quanto disco ho a disposizione.

Disco pieno: Loki e Prometheus smettono di scrivere, e si perdono proprio i dati del guasto.

Agent

Un programma che gira sulla macchina da osservare



Vede tutto da dentro.

Ma va installato e aggiornato su ogni macchina.

Alloy è il nostro agent: è lui a leggere i log e misurare il PC.

Agentless

Si osserva da fuori, senza installare niente

1. `ping its.fabersoft.net` — è acceso?
2. `curl -I http://its.fabersoft.net:3000` — il sito risponde?
3. `SNMP` — fra poco

Il troubleshooting della lezione 1 era agentless.

Vede solo quello che la macchina mostra a chi sta fuori.

Pull: il server va a chiedere

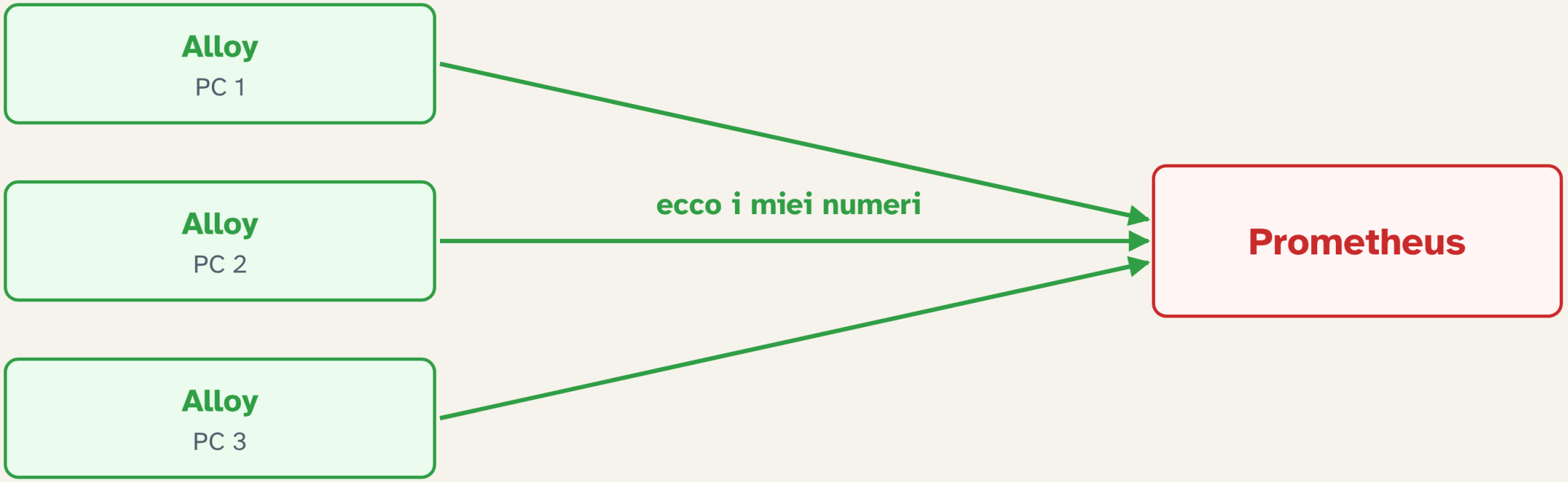
Prometheus, di base, fa così



Il server deve riuscire a raggiungere ogni macchina.

Push: è la macchina a spedire

Alloy verso Loki e verso Prometheus



Funziona anche da dietro il router di casa.

Pull o push?

Pull

+ il server sa subito chi è giù

+ si decide tutto sul server

- deve raggiungere ogni macchina

- niente IP privati

Push

+ va bene dietro un router

+ i PC non aprono porte: ascolta solo il server

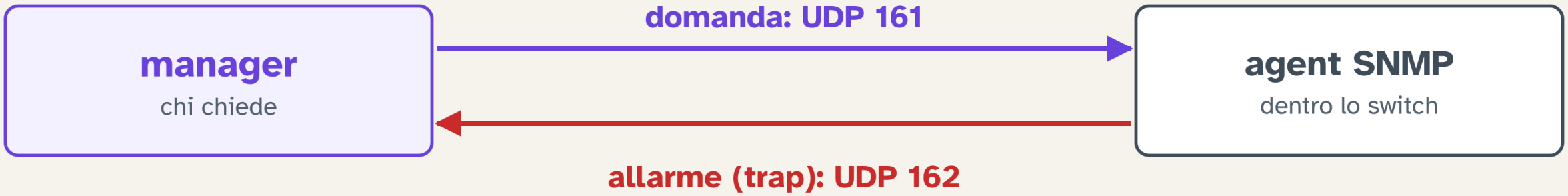
- se l'agent tace, non sai perché

- ogni PC va configurato

I vostri PC hanno IP privati: push. Come per i log in lezione 4.

SNMP

Per gli apparati dove non installi niente: switch, router, stampanti, UPS



Lo switch ha già il suo piccolo agent, messo dal costruttore.

OID e MIB

Ogni valore ha un indirizzo fatto di numeri

```
$ snmpget -v2c -c public 192.168.1.1 1.3.6.1.2.1.1.3.0
```

1.3.6.1.2.1.1.3.0

= da quanto tempo è acceso (sysUpTime)

OID: l'indirizzo numerico di un valore.

MIB: il dizionario che dà un nome ai numeri.

SNMP e sicurezza

v1, v2c

community "public"

è una password
che tutti conoscono

e viaggia in chiaro

v3

utente e password

autenticazione
e cifratura

Con tcpdump, chi è sulla stessa rete legge la community.

SNMP in pratica

La stessa domanda (da quanto è acceso l'apparato?) in v2c e in v3

v2c: basta la community, che viaggia in chiaro

```
$ snmpget -v2c -c public 192.168.1.1 1.3.6.1.2.1.1.3.0
iso.3.6.1.2.1.1.3.0 = Timeticks: (8640000) 1 day, 0:00:00.00
```

v3: utente, autenticazione (SHA) e cifratura (AES)

```
$ snmpget -v3 -l authPriv -u monitor -a SHA -A <pwd> -x AES -X <pwd> \
      192.168.1.1 1.3.6.1.2.1.1.3.0
```

1.3.6.1.2.1.1.3.0 è l'OID di sysUpTime: il tempo dall'ultimo avvio.

Con tcpdump sulla porta UDP 161 la community di v2c si legge; con v3 no.

Dalla sorgente al grafico

Lo stesso flusso, per i log e per le metriche

Raccolta collection	**Parsing** e normalizzazione	**Storage** con una retention	**Visualizzazione** e interrogazione
prende i dati dove nascono	stesso formato e nomi: etichette, unità, orari	conserva i dati per un tempo deciso	grafici, ricerche, dashboard
LOG			
rsyslog, Alloy, Beats	**Logstash, gli stage di Alloy**	**Loki, Elasticsearch**	**Grafana, Kibana**
METRICHE			
exporter, Telegraf, agent Zabbix, SNMP	**etichette host e job, unità di misura**	**Prometheus, InfluxDB**	**Grafana, Zabbix**

L'alerting parte dallo storage: è una query che gira da sola, ogni pochi secondi.

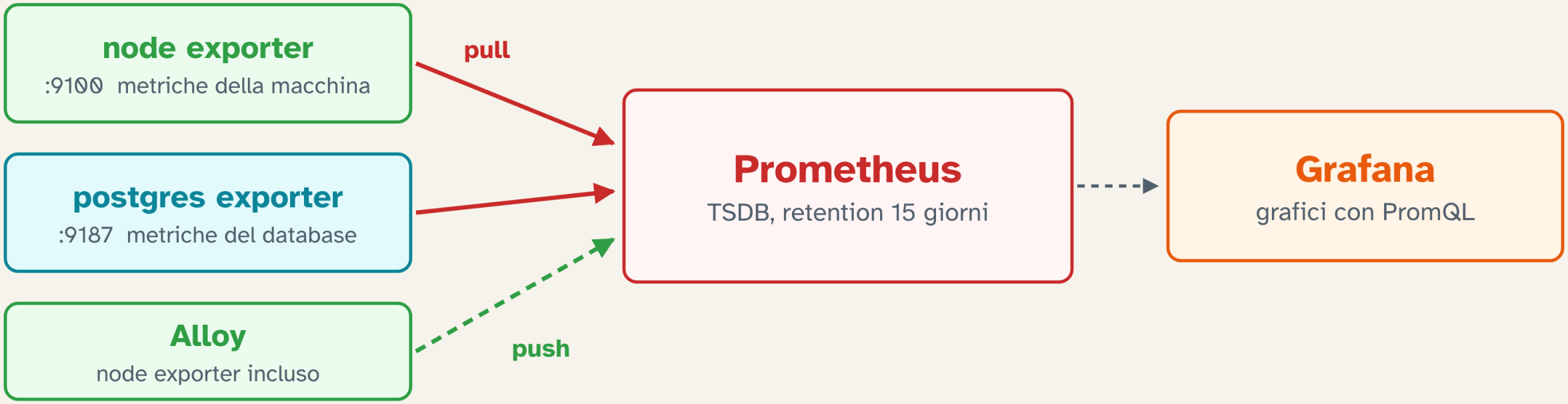
Gli strumenti di monitoraggio più usati

Nel corso usiamo Prometheus: è lo standard e si integra con Alloy e Grafana

	come raccoglie	dove salva	in una riga
Zabbix	agent proprio, e agentless: SNMP, IPMI, ping	database relazionale (PostgreSQL, MySQL)	tutto in uno: raccolta, web, trigger e allarmi
TIG Telegraf InfluxDB Grafana	Telegraf: agent con molti plugin, in push	InfluxDB, una TSDB	come il nostro stack, ma in push nativo
Prometheus	pull da /metrics degli exporter; push con remote write	TSDB propria	PromQL e Alertmanager; lo standard per i container
Netdata	agent su ogni macchina, una metrica al secondo	sulla macchina stessa (o inoltra a un parent)	zero configurazione, dashboard già pronta
PCP Performance Co-Pilot	demone pmcd, un agent per sottosistema	archivi di pmlogger	analisi delle prestazioni, anche a posteriori

Prometheus

Un database a serie temporali (TSDB) che si interroga con PromQL




```
up                                                          1 se il target risponde
node_load1                                                  carico medio dell'ultimo minuto
100 - avg by (host) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100   CPU usata in %, per host
```

Il server: arriva Prometheus

Un servizio in più in vps-corso/compose.yml

```yaml
prometheus:
  image: prom/prometheus:v3.15.0
  command:
    - --config.file=/etc/prometheus/prometheus.yml
    - --storage.tsdb.path=/prometheus
    - --storage.tsdb.retention.time=15d
    - --web.enable-remote-write-receiver
  volumes:
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prometheus-data:/prometheus
  ports:
    - "9090:9090"
  restart: unless-stopped
```

← retention

← accetta il push

← il database

← web e push

In fondo al file, tra i volumes, una riga in più: prometheus-data:

Due righe da capire

```
--storage.tsdb.retention.time=15d
```

Tiene 15 giorni di metriche, poi butta i più vecchi.

```
--web.enable-remote-write-receiver
```

Accetta i dati spediti dagli Alloy (push).

Senza questa riga Prometheus sa solo chiedere (pull).

Il client: Alloy fa da node exporter

Tre blocchi in più in pc-studente/alloy/config.alloy

```
prometheus.exporter.unix "pc" {
  procfs_path = "/host/proc"
  sysfs_path  = "/host/sys"
}

prometheus.scrape "pc" {
  targets    = prometheus.exporter.unix.pc.targets
  forward_to = [prometheus.remote_write.vps.receiver]
}

prometheus.remote_write "vps" {
  endpoint {
    url = "http://its.fabersoft.net:9090/api/v1/write"
  }
  external_labels = { host = constants.hostname }
}
```

1. misura il PC

2. raccogli

ogni 15 s

3. spedisci

a Prometheus,

con l'etichetta host

Il client: due volumi in più

In pc-studente/compose.yml, sotto volumes

```
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
```

Alloy legge da lì CPU, memoria, disco e rete.

Su Windows (WSL2 o Docker Desktop):

misurate la macchina virtuale Linux, non Windows.

Riavvio e verifica

1. `cd its-jesi-lessons/logging/pc-studente`
2. `docker compose up -d` — ricrea Alloy
3. `http://its.fabersoft.net:9090` — Prometheus
4. `up` — chi sta spedendo?
5. `node_load1{host="mario-rossi"}` — il tuo PC

Il primo pannello in Grafana

Dashboards → New → sorgente dati Prometheus

```
100 * (1 - avg by (host) (
  rate(node_cpu_seconds_total{mode="idle"}[1m])))
```

= la percentuale di CPU usata, una linea per ogni studente

idle = il tempo in cui la CPU non fa niente.

100 meno il tempo libero = il tempo occupato.

Metriche, eventi e alerting

Metrica
un numero nel tempo

`CPU all'87% alle 10:15`

Evento
un fatto puntuale

`10:15:02 servizio riavviato`

Alert
una situazione anomala

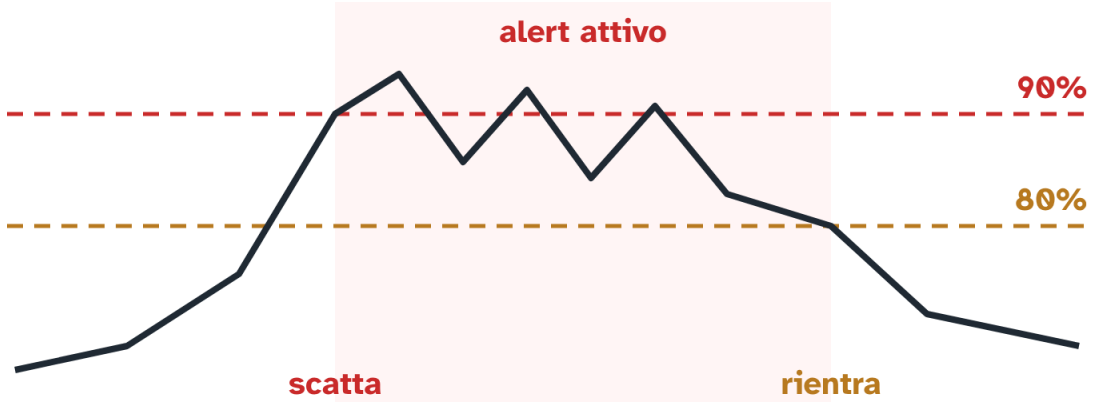
`CPU oltre il 90% da 5 minuti`

Soglie (thresholds)

statica: un valore fisso, es. disco oltre il 90%

dinamica: rispetto al normale, es. il triplo del solito

Isteresi: scatta sopra il 90%, rientra solo sotto l'80%



Senza isteresi si accenderebbe e spegnerebbe a ogni oscillazione.

Severity

Info	da registrare, nessuna azione
Warning	da guardare prima che peggiori
High	da risolvere presto
Critical	servizio fermo: subito

Alert fatigue

troppi falsi positivi: nessuno legge più gli alert, nemmeno quello vero.

Un alert solo se serve un'azione.

OBSERVABILITY

Logging + **Monitoring** + **Troubleshooting** = **Observability**

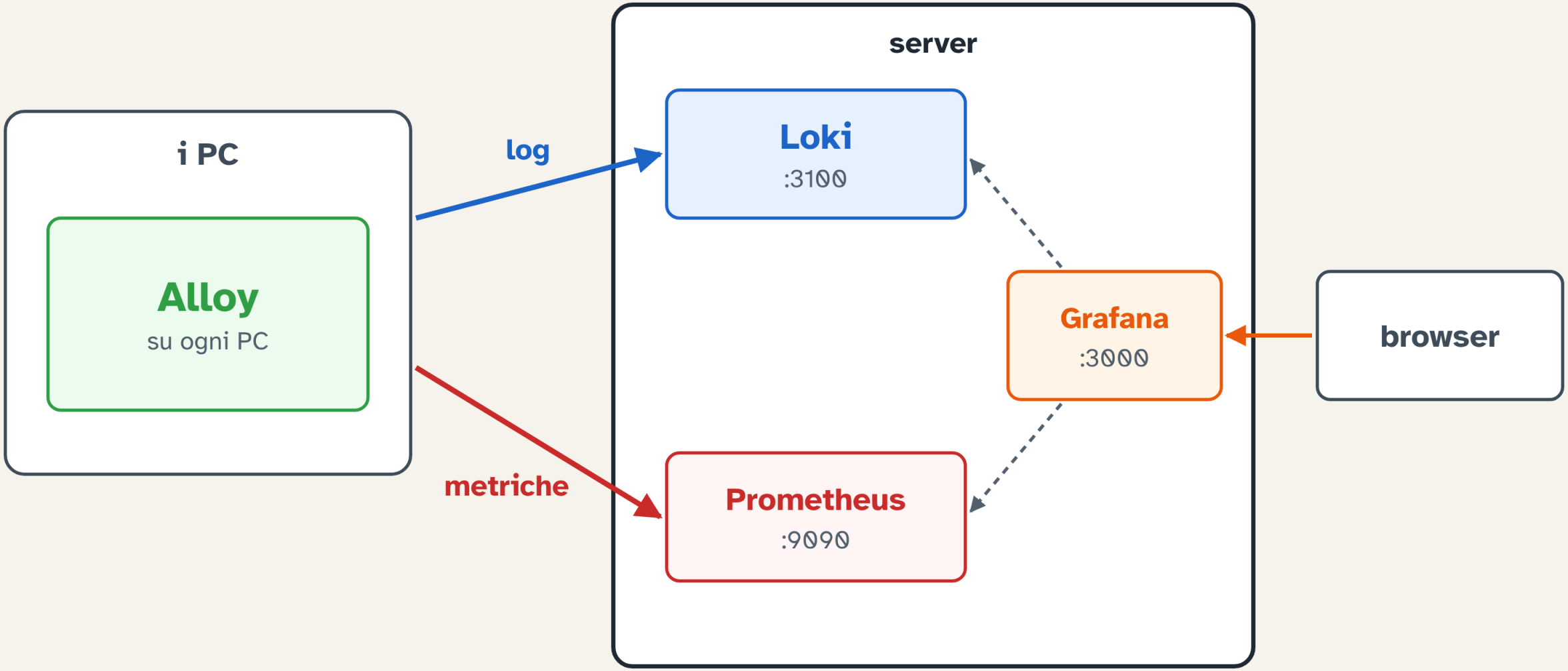
Capire cosa succede dentro un sistema guardando quello che produce:
log ed eventi, metriche (e, nelle architetture più grandi, le tracce).

- sapere in ogni momento cosa succede nella rete e nei servizi
- correlare: stesso istante, stesso host, log e metriche affiancati in Grafana
- arrivare alla root cause di un guasto in tempi rapidi

Il monitoring ti dice che qualcosa non va; l'observability ti aiuta a capire perché.

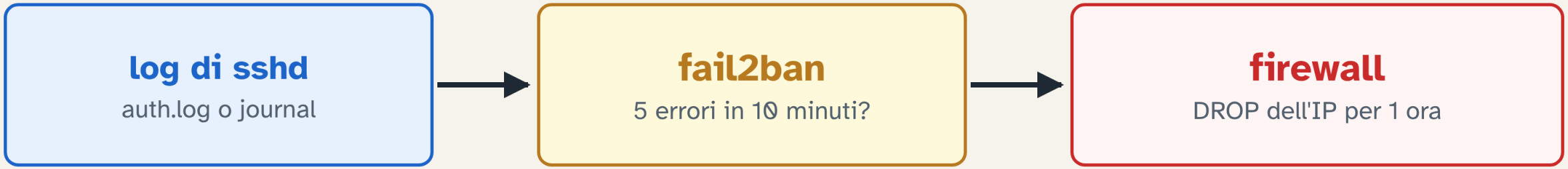
Il quadro finale

Lo stesso su befair3 e sul VPS del corso



fail2ban: dal log all'azione

Legge i log e banna chi sbaglia troppe volte




```
# /etc/fail2ban/jail.local
[sshd]
enabled  = true
maxretry = 5
findtime = 10m
bantime  = 1h
```

```
$ sudo fail2ban-client status sshd
   |- Currently banned: 1
   `- Banned IP list: 45.9.14.207
```

Nel nostro log il brute force parte con 8 errori di fila:

sarebbe stato bannato prima dell'accesso riuscito.

Lezione 2: "fail2ban" era il nome finto del miner. Controllate sempre il percorso del programma.

Il firewall che scrive nel log

iptables -j LOG: ogni pacchetto che corrisponde finisce nel log del kernel

```
$ sudo iptables -I INPUT -p tcp --dport 22 -j LOG --log-prefix "SSH-IN: "
$ sudo journalctl -k -f | grep SSH-IN
Oct 28 10:15:02 srv kernel: SSH-IN: IN=eth0 OUT= SRC=45.9.14.207 DST=203.0.113.10
    ... PROTO=TCP SPT=42216 DPT=22 SYN
```

- SRC e DST sono il livello 3, SPT e DPT il livello 4
- -j LOG non blocca: scrive e passa alla regola successiva
- -I la mette in cima, prima delle regole che accettano o scartano
- Sui sistemi recenti c'è nftables: stessa idea, sintassi diversa

Prima di uscire

1. **Che differenza c'è tra un log e una metrica?**
2. **Pull o push: cosa usiamo, e perché?**
3. **Cosa vuol dire retention di 15 giorni?**
4. **Quando uso SNMP?**

Ora: simulazione del compito

La prossima lezione: ripasso e compito.